Here are **High-Yield Technical Interview Notes** designed for quick revision and "whiteboard" readiness. These cover the specific intersection of **Java Backend** and **Cyber Security/Linux** that Quest Global is looking for.

📝 Part 1: Java & Coding "Cheat Sheet"

1. The "Must-Know" Code Snippets

- **Singleton Pattern (Thread-Safe):**

Java

```
public class DatabaseConnector {
  // Volatile ensures visibility across threads
  private static volatile DatabaseConnector instance;

  private DatabaseConnector() {} // Private constructor

  public static DatabaseConnector getInstance() {
    if (instance == null) {
      synchronized (DatabaseConnector.class) {
        if (instance == null) { // Double-checked locking
          instance = new DatabaseConnector();
        }
      }
    }
    return instance;
  }
}
```

- **Immutable Class (Security Best Practice):**
    - *Why:* Immune to race conditions; good for caching and keys.
    - *How:* final class, private final fields, no setters, deep copy mutable fields in constructor.

Java

```
public final class SecureConfig {
   private final String apiKey;
   private final List<String> allowedIps;

   public SecureConfig(String apiKey, List<String> ips) {
      this.apiKey = apiKey;
      this.allowedIps = new ArrayList<>(ips); // Defensive Copy
   }
   public List<String> getAllowedIps() {
      return new ArrayList<>(allowedIps); // Return copy, not reference
   }
}
```

2. Java 8 Streams (Data Processing)

- *Scenario:* "Filter a list of users to find 'Admins' and get their emails."

```
Java
List<String> adminEmails = users.stream()
   .filter(u -> "ADMIN".equals(u.getRole()))
   .map(User::getEmail)
   .collect(Collectors.toList());
```

3. HashMap Internals (Classic Question)

- **Key Concepts:**
    - **Hashing:** key.hashCode() determines the bucket index.
    - **Collision:** If two keys land in the same bucket, they form a **Linked List**.
    - **Java 8 Change:** If the list grows > 8 items, it converts to a **Red-Black Tree** (O(log n)) to prevent DoS attacks (Hash Flooding).
    - **Contract:** If you override equals(), you **must** override hashCode().

🐧 **Part 2: Linux Shell Scripting (Whiteboard Ready)**

1. Critical Commands

- **Find large files:** find / -type f -size +100M
- **Search logs for error:** grep -r "ERROR" /var/log/myapp/
- **Count occurrences:** grep "404" access.log | wc -l
- **Check Disk Space:** df -h (Disk Free), du -sh * (Disk Usage of current folder).
- **Check Ports:** netstat -tulpn | grep LISTEN

2. "Write a Script" Scenarios

- *Scenario:* "Write a script to backup a folder and delete files older than 7 days."

Bash

```
#!/bin/bash

SOURCE="/var/www/html"

DEST="/backup/www"

DATE=$(date +%Y-%m-%d)


# 1. Create Tarball

tar -czf $DEST/backup-$DATE.tar.gz $SOURCE


# 2. Delete old backups (Modified time > 7 days)

find $DEST -name "*.tar.gz" -mtime +7 -delete


echo "Backup completed for $DATE"
```

3. File Permissions (Octal math)

- **Recall:** Read=4, Write=2, Execute=1.
- **755:** Owner(7=rwx), Group(5=rx), Others(5=rx). (Standard for scripts/binaries).
- **644:** Owner(6=rw), Group(4=r), Others(4=r). (Standard for config files).
- **600:** Owner(6=rw), Group(0), Others(0). (Standard for **SSH Keys**).

🔐 **Part 3: Cyber Security Concepts (Rapid Fire)**

1. Hashing vs. Encryption vs. Encoding

- **Encoding (Base64):** Reversible. NO SECURITY. (Just format translation).
- **Encryption (AES/RSA):** Reversible *with a key*. (Confidentiality).
- **Hashing (SHA-256):** One-way. Irreversible. (Integrity/Passwords).
    - *Salt:* Random data added to password before hashing to prevent **Rainbow Table** attacks.

2. SSL/TLS Handshake (Simplified)

1. **Client Hello:** "I support TLS 1.3, here are my cipher suites."
2. **Server Hello:** "Let's use TLS 1.3. Here is my **Certificate** (Public Key)."
3. **Verification:** Client checks if Certificate is valid (signed by a trusted CA).
4. **Key Exchange:** They generate a **Shared Session Key** (Symmetric Key) using the Public Key.
5. **Encrypted Tunnel:** All data is now encrypted with the Session Key (AES).

3. Common Attacks & Defenses

- **CSRF (Cross-Site Request Forgery):**
    - *Attack:* User visits malicious site -> Site makes invisible request to bank.com/transfer.
    - *Defense:* **Anti-CSRF Tokens** (Hidden field in forms) or SameSite Cookie attribute.
- **Man-in-the-Middle (MitM):**
    - *Attack:* Intercepting Wi-Fi traffic.
    - *Defense:* **HTTPS (TLS)** + **HSTS** (Strict Transport Security - forces browser to use HTTPS).

🧠 **Part 4: System Design (Security Focus)**

Design a Secure Login System

1. **Transport:** HTTPS only.
2. **Storage:** Hash passwords using **BCrypt** or **Argon2** (never MD5/SHA1).
3. **Rate Limiting:** Block IP after 5 failed attempts (prevent Brute Force).

4. **Response:** Generic error message ("Invalid email or password") to prevent Username Enumeration.
5. **Session:** Return a **JWT** (with short expiry) or a **HttpOnly Cookie** (prevent XSS access).

❓ Part 5: Questions to Ask the Interviewer

Asking good questions shows you are a senior-level thinker.

1. "How does Quest Global handle 'Shift Left' security? are developers involved in configuring the Nessus scans?"
2. "What is the biggest security challenge the team faced in the last 6 months?"
3. "Does the team manage its own CI/CD pipelines, or is there a dedicated DevOps team?"
4. "What version of Java/Spring Boot is the core platform currently running on?"

💡 Final "Golden Rule" for the Interview

If you don't know the answer to a security question:

Do NOT guess. In security, guessing is dangerous. **Say:** *"I haven't encountered that specific tool, but my approach would be to consult the official documentation/OWASP guidelines and verify in a sandbox environment before applying it to production."*